

Informe de Análisis de Arquitectura de Software y Evaluación Técnica - Proyecto HeartCoin

Clasificación: Confidencial - Uso Interno
Versión: 1.0
Autor: Analista de Sistemas y Documentación Técnica

1. Introducción

El presente documento constituye un informe de análisis exhaustivo y evaluación técnica global del ecosistema de software **HeartCoin**, el cual comprende un backend monolítico modularizado en Node.js/Express y un cliente móvil de alta fidelidad desarrollado en Flutter/Dart. Este análisis evalúa de forma holística la arquitectura, los contratos de interfaz (API REST), la integridad de la base de datos relacional, la seguridad transaccional, y la viabilidad técnica y de experiencia de usuario (UX) para la transición segura de un estado de prototipo funcional avanzado hacia un entorno de producción regulado.

2. Objetivo del Análisis

El objetivo principal es auditar, estructurar y documentar las especificaciones y riesgos de la arquitectura desacoplada de HeartCoin. Se busca identificar fallas críticas latentes, evaluar la consistencia del contrato de datos entre el cliente y el servidor, verificar el cumplimiento de estándares de seguridad (como control de accesos, manejo de tokens y validaciones antifraude) y proveer una base técnica formal que guíe las decisiones de ingeniería y el plan de mitigación antes del lanzamiento oficial.

3. Alcance del Análisis

La evaluación estática y el análisis de diseño comprenden las siguientes dimensiones de ambos repositorios:

Componente	Módulos / Rutas Analizadas	Artefactos Evaluados
Backend (Node.js)	backend/src/config, controllers, middlewares, routes, services, database	Configuración de entorno, ciclo de vida de peticiones Express, lógica transaccional de billeteras e iniciativas, esquema relacional MariaDB (11 tablas).

Componente	Módulos / Rutas Analizadas	Artefactos Evaluados
Frontend (Flutter)	lib/config, lib/shared/services, lib/shared/models, lib/onboarding, lib/home	Estrategia de inmutabilidad, gestión de estado con Riverpod, consumo HTTP vía Dio, flujos de autenticación y vistas de usuario.

4. Descripción General del Sistema

HeartCoin se define como una plataforma financiero-social descentralizada o regulada internamente que incentiva la participación comunitaria. Sus operaciones principales se dividen en cuatro macro-dominios funcionales:

- **Autenticación y Sesión (Auth):** Registro, login multi-identificador (email o username) y persistencia segura de sesión mediante arquitectura de doble token.
- **Iniciativas Comunitarias (Initiatives):** Creación, búsqueda, votación y participación activa de usuarios, organizaciones y empresas en proyectos sociales.
- **Módulo Social (Social):** Publicación de contenido multimedia (posts, stories) enfocado en evidenciar el impacto social de las iniciativas.
- **Billetera Digital (Wallet):** Inicialización de billeteras, consulta de balances, traza criptográfica de transacciones y reclamación de recompensas validadas por un sistema externo antifraude (IaaS).

5. Metodología de Análisis

Se aplicó una metodología de revisión de arquitectura de software basada en escenarios y análisis estático de documentación y estructuras de código fuente:

1. **Inspección Estática Basada en Capas:** Verificación de dependencias y aislamiento de responsabilidades (Separation of Concerns).
2. **Auditoría de Contratos de Interfaz:** Comparación de las firmas de endpoints documentadas en archivos de diseño contra la implementación real en enrutadores y controladores.
3. **Análisis de Modos de Falla Criptográfica y Transaccional:** Evaluación del comportamiento de las funciones críticas de negocio ante fallas de infraestructura o ataques concurrentes.

6. Hallazgos y Análisis Técnico/Funcional

6.1. Capa de Configuración e Infraestructura

La capa de configuración del backend exhibe una alta centralización en `environment.js`, garantizando un control adecuado del arranque del servidor. Sin embargo, coexiste una duplicación en la fuente de verdad para la conexión a la base de datos entre `database.js` y `environment.js`. En el frontend, la clase `AppConfig` centraliza las variables utilizando

constructores semánticos de Dart, aislando exitosamente los entornos de compilación.

6.2. Capa de Base de Datos y Persistencia

El esquema relacional implementado en MariaDB 10.4 está compuesto por 11 tablas (7 del núcleo principal y 4 pertenecientes a la extensión social). El diseño de restricciones de llave foránea (FK) es técnicamente robusto, utilizando políticas de borrado e integridad referencial coherentes (como ON DELETE CASCADE en registros de sesión y SET NULL en bitácoras de auditoría para preservación de histórico). No obstante, se detectó una anomalía de diseño estructural en la tabla initiatives: el campo organization_id carece de una llave foránea asociada, lo que permite la inserción de identificadores huérfanos.

6.3. Capa de Servicios y Lógica de Negocio

Los servicios del backend implementan abstracciones limpias mediante el uso de librerías como bcryptjs, crypto, y axios. El servicio de billetera (WalletService) destaca por su acoplamiento ordenado con BlockchainService para firmar transacciones criptográficamente. Sin embargo, el subsistema clave para la viabilidad financiera del producto, laaSService (encargado de la validación antifraude para la entrega de tokens), se encuentra operando bajo un comportamiento simulado (**Mock Mode**).

6.4. Capa de Interfaz de API (Rutas y Controladores)

El backend expone un total de 29 endpoints controlados a través de Express Router y protegidos por la librería express-rate-limit. El mapeo de rutas muestra un desacoplamiento claro. A pesar de esto, existen divergencias de convención nominal e inconsistencias en las respuestas HTTP devueltas por los middlewares de control de acceso.

6.5. Capa de Modelos y Servicios del Cliente Móvil

El cliente Flutter implementa el cliente HTTP Dio de forma avanzada, integrando interceptores para la inyección automática de cabeceras de autorización y renovación de tokens. Los modelos de datos en lib/shared/models aplican inmutabilidad estricta y deserialización segura defensiva. El hallazgo principal aquí radica en la redundancia estructural entre las clases WalletBalance y WalletStats, las cuales procesan datos similares de formas paralelas.

7. Problemas Detectados y Riesgos Identificados

7.1. Riesgos Críticos (Bloqueadores de Lanzamiento)

- **laaSService en Modo Mock con Falla Probabilística:** El servicio antifraude emula un entorno productivo rechazando de manera aleatoria y no determinista el 5% de las solicitudes de reclamación de recompensas. De desplegarse así en producción, generará un falso positivo del 5% en fallas del sistema, destruyendo la confianza del usuario y corrompiendo el modelo de fidelización de HeartCoin.
- **Ausencia de Validación de Producción para IAAS_API_URL:** La capa de configuración del servidor no valida la presencia o estructura de la URL del servicio laaS

en producción. Si la variable de entorno se omite en el servidor físico, el backend mutará silenciosamente al uso del mock de desarrollo de manera automática.

- **Contenido Vacío en Páginas de Onboarding:** El flujo de bienvenida del cliente móvil posee pantallas intermedias carentes de contenido real o textos finales. Un flujo de onboarding incompleto bloquea la adopción de usuarios y proyecta baja calidad de software.
- **Ausencia de Restauración de Sesión en SplashScreen:** La lógica del archivo Splash del frontend obliga a todos los usuarios a realizar el flujo de onboarding completo cada vez que abren la aplicación, ignorando si cuentan con una sesión activa y con tokens válidos guardados localmente.

7.2. Riesgos Altos y Deuda Técnica Media

- **Inconsistencia de Códigos de Error HTTP en Middlewares:** Cuando un token JWT es inválido o corrupto, el middleware authMiddleware responde con un código 403 Forbidden en lugar de un estándar 401 Unauthorized. Esto confunde a los interceptores del frontend (como el de Dio), impidiendo la activación correcta de la rutina de refresco de tokens.
- **Ausencia de Transacciones Criptográficas y de BD en register():** El método de registro en auth_service.js realiza inserciones secuenciales en múltiples tablas sin abrir una transacción SQL formal. Si ocurre una desconexión a mitad del proceso, la base de datos quedará en un estado inconsistente (ej. usuario creado sin sus tokens iniciales o billetera asociada).
- **Falta de Enums en Campos de Dominio Cerrado (Frontend):** Campos críticos como el role de la entidad AuthUser se manejan como strings planos en la interfaz. Una discrepancia de caracteres (ej. "admin" vs "Admin") puede saltarse las validaciones visuales de la UI o provocar fallas en tiempo de ejecución.

8. Fortalezas del Sistema

- **Arquitectura Guiada por Principios de Inmutabilidad:** El frontend adopta la inmutabilidad de datos de manera transversal, reduciendo efectos secundarios y bugs asociados a la mutación de estado en la UI.
- **Diseño Defensivo en Deserialización:** El cliente Flutter cuenta con mecanismos de parseo JSON robustos que previenen caídas de la aplicación ante cambios inesperados en los tipos de datos enviados por el servidor.
- **Excelente Control de Integridad Financiera Interna:** La combinación de WalletService con una bitácora inmutable firmada (BlockchainService) y un sistema estructurado de logs de auditoría representa un estándar alto de desarrollo financiero.

9. Recomendaciones y Plan de Mitigación

Para corregir la deuda técnica identificada y asegurar la estabilidad de la arquitectura desacoplada, se definen las siguientes acciones prioritarias:

1. **R-01: Integración Real de IaaS y Validación de Entorno (Prioridad: Crítica)**
Sustituir de forma definitiva la simulación del servicio antifraude por la API de producción del proveedor IaaS. Modificar environment.js para exigir de manera obligatoria e

inderogable la existencia de la variable IAAS_API_URL cuando NODE_ENV === 'production'.

2. **R-02: Consolidación de Modelos de Billetera y Enums en Frontend (Prioridad: Alta)**
Unificar las clases de datos WalletBalance y WalletStats en un único modelo de dominio cohesivo para evitar divergencias visuales en el balance del usuario. Transformar los campos de estado y roles de strings planos a Enums nativos de Dart.
3. **R-03: Implementación de Transacciones SQL en Servicios Críticos (Prioridad: Alta)**
Refactorizar los métodos de escritura múltiple del backend (especialmente el registro de usuarios y la transferencia de activos) para envolver las consultas dentro de bloques de transacción de mysql2/promise (await connection.beginTransaction()), garantizando la propiedad ACID de la base de datos.
4. **R-04: Corrección de Flujo HTTP 401 y Restauración de Sesión (Prioridad: Alta)**
Modificar auth.middleware.js en el backend para retornar código 401 Unauthorized con el flag de token inválido. Paralelamente, implementar la llamada al servicio de sesión en el SplashScreen de Flutter para redirigir directamente al usuario al módulo Home si la sesión local permanece vigente.

10. Conclusión

El ecosistema del proyecto HeartCoin demuestra un nivel de diseño técnico **Bueno-Sólido**, caracterizado por una clara separación de responsabilidades y la adopción de librerías modernas de la industria de desarrollo web y móvil. El sistema califica actualmente como un prototipo funcional avanzado de alto rendimiento. Con la implementación del plan de mitigación propuesto (enfocado en desactivar los comportamientos mock, unificar las estructuras de datos redundantes y estandarizar el protocolo de comunicación de errores), la arquitectura alcanzará un nivel de madurez **Excelente**, apto para operar de forma segura, escalable y confiable en un entorno de producción real.